
On the Complexity of Scheduling with Batch Setup Times

Author(s): Clyde L. Monma and Chris N. Potts

Source: *Operations Research*, Vol. 37, No. 5 (Sep. - Oct., 1989), pp. 798-804

Published by: INFORMS

Stable URL: <http://www.jstor.org/stable/171025>

Accessed: 11/03/2009 06:56

Your use of the JSTOR archive indicates your acceptance of JSTOR's Terms and Conditions of Use, available at <http://www.jstor.org/page/info/about/policies/terms.jsp>. JSTOR's Terms and Conditions of Use provides, in part, that unless you have obtained prior permission, you may not download an entire issue of a journal or multiple copies of articles, and you may use content in the JSTOR archive only for your personal, non-commercial use.

Please contact the publisher regarding any further use of this work. Publisher contact information may be obtained at <http://www.jstor.org/action/showPublisher?publisherCode=informs>.

Each copy of any part of a JSTOR transmission must contain the same copyright notice that appears on the screen or printed page of such transmission.

JSTOR is a not-for-profit organization founded in 1995 to build trusted digital archives for scholarship. We work with the scholarly community to preserve their work and the materials they rely upon, and to build a common research platform that promotes the discovery and use of these resources. For more information about JSTOR, please contact support@jstor.org.



INFORMS is collaborating with JSTOR to digitize, preserve and extend access to *Operations Research*.

ON THE COMPLEXITY OF SCHEDULING WITH BATCH SETUP TIMES

CLYDE L. MONMA

Bell Communications Research, Morristown, New Jersey

CHRIS N. POTTS

University of Southampton, Southampton, England

(Received December 1986; revision received March 1988; accepted July 1988)

Many practical scheduling problems involve processing several batches of related jobs on common facilities where a setup time is incurred whenever there is a switch from processing a job in one batch to a job in another batch. We extend various scheduling models to include batch setup times. The models include the one-machine maximum lateness, total weighted completion time, and number of late jobs problems. In all these cases, a dynamic programming approach results in an algorithm that is polynomially bounded in the number of jobs, but is exponential in the number of batches. We also study the parallel machine model with preemption and show that the maximum completion time, maximum lateness, total weighted completion time, and number of late jobs problems are NP-hard, even for the case of two identical parallel machines, and sequence independent setup times.

Many practical scheduling problems involve processing several batches of related jobs on common facilities where a setup time is incurred whenever there is a switch from processing a job in one batch to a job in another batch. The setup times may depend on the batch of the previous job as well as that of the current job. For example, flexible manufacturing systems can produce several different types of products on the same machinery, but may require a setup time for rearranging or retooling workstations when there is a switch in product type. We are motivated primarily by situations where a large number of jobs are grouped into a relatively small number of batches.

To indicate the importance of scheduling with batch setup times, three practical examples are given. The first example, described in Conway, Maxwell and Miller (1967), involves the production of different colors of paint on the same machine. A setup time for cleaning the machine is incurred whenever there is a color change. The thoroughness necessary in cleaning the machine depends on both the color being removed and the color for which the machine is being prepared. The problem of completing production in the minimum time is related to the traveling salesman problem (Lawler et al. 1985).

A second example is the scheduling of computer

systems. In this problem, a collection of tasks is to be processed on a computer where each task has a processing time, a deadline for completion, and a requirement for a particular compiler to be resident in the computer's memory. If the appropriate compiler is resident, then the task may start processing immediately; otherwise a setup time is incurred to bring the relevant compiler into memory. In this example, the setup time depends only on the time to load the compiler for the current job, and does not depend on the previous job. Deadline scheduling problems with sequence-independent setup times are studied in Bruno and Downey (1978), who derive computational complexity results.

A final example is motivated by situations where the labor force is a limiting resource and workers must be switched from machine to machine, incurring a setup time, in order to complete all tasks. The problem of one worker switching between tasks on two machines in order to minimize the total completion time is studied in Sahney (1972), where a branch-and-bound algorithm is presented.

To state our problem of scheduling with batch setup times more precisely, we are given N jobs that are divided into B batches. Each batch b , for $1 \leq b \leq B$, contains N_b jobs which are arbitrarily labeled the 1st, 2nd, . . . , N_b th. All jobs are available for processing at

Subject classifications: Analysis of algorithms. Dynamic programming. Production/scheduling.

time zero. We let $p_{ib} > 0$ denote the *processing time* of the i th job in batch b . A *setup time* $s_{bc} \geq 0$ is incurred whenever a job in batch c is processed immediately after a job in batch b . We have $s_{bb} = 0$ for all batches b . Also, an initial setup time s_{0b} is incurred if a job from batch b is processed first on a machine. The initial setup can be treated in the same way as any other setup by regarding the machine as initially setup for batch 0. We make the reasonable assumption that setup times satisfy the *triangle inequality*, that is $s_{ac} \leq s_{ab} + s_{bc}$ for all batches a, b, c , including the case $a = 0$. The setup times are *sequence independent* if $s_{bc} = s_c$ for $0 \leq b \leq B$ and $1 \leq c \leq B$, where $b \neq c$. We can regard the setup times as representing a setup job. Each of the original jobs must either immediately follow another job in its batch or the appropriate setup job. The jobs are to be scheduled on M identical parallel machines with setup jobs inserted as necessary. Although preemption of jobs is allowed, clearly for any “regular” optimality criteria (Conway, Maxwell and Miller), it is of no advantage for the one-machine case.

In Section 1, we study the one-machine problem with various optimality criteria. We show that once the order of jobs within each batch is known, a dynamic programming algorithm can be used to optimally merge the ordered batches into a single schedule; the time required is polynomial in the number of jobs but exponential in the number of batches. It is easy to derive an optimal order within each batch for the maximum lateness, total weighted completion time, and the number of late jobs problems. For any fixed number of batches, therefore, the problem is, in theory, efficiently solvable. The exponential time-bound is to be expected for the maximum lateness and the number of late jobs problems because they are shown to be NP-hard when the number of batches is allowed to be variable, even when the setup times are sequence-independent (Bruno and Downey). Also, our method provides a polynomial time-bound for the total completion time problems in which two batches are scheduled on one machine; this case was solved previously by the branch-and-bound method with no polynomial time-bound given (Sahney). In Section 2, we consider the scheduling problem on identical parallel machines with preemption allowed. We show that the maximum completion time, maximum lateness, total weighted completion time, and maximum number of late jobs problems are all NP-hard, even for the case of only two machines and sequence-independent setup times. Some concluding remarks are contained in Section 3.

1. ONE-MACHINE SCHEDULING

In this section, we show that many scheduling problems with batch setup times can be solved in two stages. First, the order of the jobs within each batch is fixed. Then, in the second stage, the ordered batches are merged into an overall schedule using a dynamic programming algorithm. Special attention is given to the one-machine maximum lateness, total weighted completion time, and number of late jobs problems.

For any schedule, we define C_{ib} to be the *completion time* of our arbitrarily labeled i th job in each batch b . Each such job has a cost function $f_{ib}(\cdot)$ associated with it. A *sum criterion* evaluates a schedule by assigning it a cost of $\sum_{i,b} f_{ib}(C_{ib})$. A *max criterion* evaluates a schedule by assigning it a cost of $\max_{i,b} f_{ib}(C_{ib})$.

The cost function for the i th job of each batch b may depend upon a *weight* $w_{ib} \geq 0$ and/or a *due date* d_{ib} . The *maximum lateness problem* is a max criterion with a job cost function $f_{ib}(C_{ib}) = C_{ib} - d_{ib}$. The special case where all due dates are zero is called the *maximum completion time problem*. The *total weighted completion time problem* is a sum criterion with job cost function $f_{ib}(C_{ib}) = w_{ib}C_{ib}$. The *weighted number of late jobs problem* is a sum criterion with job cost function

$$f_{ib}(C_{ib}) = \begin{cases} w_{ib} & \text{if } C_{ib} > d_{ib} \\ 0 & \text{otherwise} \end{cases}.$$

When a job is completed at or before its due date it is *on time*; otherwise it is *late*. When all of the weights equal one, the previous two problems become the *total completion time* and *number of late jobs* problems, respectively.

For the maximum completion time problem, we observe that since the triangle inequality for setup times holds, there exists an optimal schedule in which all jobs from the same batch are sequenced contiguously in any order. The problem, therefore, reduces to one of scheduling the batches so that the total setup time is minimized. For sequence-independent setup times any ordering of the batches is optimal, whereas, for the general case, an optimal ordering is obtained from the solution of the traveling salesman problem defined by the matrix of cost elements s_{bc} for $0 \leq b, c \leq B$, where $s_{b0} = 0$. Hence, it is clear that the maximum completion time problem is NP-hard when the number of batches is variable. The maximum lateness and number of late jobs problems are NP-hard when the number of batches is variable, even when the setup times are sequence-independent (Bruno and Downey).

We will concentrate on the maximum lateness, total weighted completion time and weighted number of late jobs problems. First, results are derived that determine the order of jobs within each batch.

Theorem 1

a. For the maximum lateness problem, there is an optimal schedule where the jobs within each batch are ordered by the earliest due date (EDD) rule.

b. For the total weighted completion time problem, there is an optimal schedule where the jobs within each batch are ordered by the shortest weighted processing time (SWPT) rule.

Proof. Consider an optimal schedule of the form $S = (D, j, E, i, F)$ where jobs i and j are from the same batch b , and D , E and F represent arbitrary (partial) sequences of jobs. We want to show that if jobs j and i are out of preference order, that is, $d_{ib} < d_{jb}$ for the maximum lateness problem, or $p_{ib}/w_{ib} < p_{jb}/w_{jb}$ for the total weighted completion time problem, then one of the schedules $S' = (D, i, j, E, F)$ or $S'' = (D, E, i, j, F)$ is an alternative optimal schedule. Repeated application of such interchanges of adjacent (partial) sequences of jobs yields the theorem. We note that the triangle inequality for the setup times ensures that the interchange which generates S' or S'' produces no additional setup time in either schedule. Therefore, we can ignore setup jobs and batches by assigning each setup job an arbitrarily large due date for the lateness problem and a zero weight for the weighted completion time problem and treat it as any other job. This simplification allows us to concentrate solely on whether the interchanges that generate the sequences S' or S'' preserve optimality.

It is useful to replace the (partial) sequence of jobs E by an equivalent composite job e . For the maximum lateness problem, a composite job h for a sequence of two jobs (i, j) is defined by $p_h = p_i + p_j$ and $d_h = \min\{d_j, d_i + p_j\}$, and for the total weighted completion time problem it is defined by $p_h = p_i + p_j$ and $w_h = w_i + w_j$. This definition easily can be extended to longer (partial) sequences of jobs. Substituting a composite job for a (partial) sequence does not affect the overall cost of any schedule for the maximum lateness problem. For the total weighted completion time problem such a substitution alters the overall cost by a constant, and thus, the preference between schedules is left unaltered.

Now it is easy to see by the well known "adjacent pairwise interchange" rule that since i is preferred to j , then either e is preferred to j in which case S'' is

optimal, or i is preferred to e in which case S' is optimal.

Note that the proof of Theorem 1 only requires that if two jobs in the same batch are out of "preference" order in an optimal schedule, then an "adjacent sequence interchange" (ASI) can be performed, which does not increase the cost and results in job i before job j . In fact, several other problems satisfying this ASI property are studied in Monma and Sidney (1979, 1987) and Theorem 1 applies to these problems as well; these include the one-machine total weighted exponential completion time problem, the least cost fault-detection problem and the two-machine maximum completion time flow shop problem.

For the weighted number of late jobs problems, it is only the order of on-time jobs that is important; the late jobs may be ordered arbitrarily after the on-time jobs. We show later how to order the on-time jobs. The proof of Theorem 1a implies the following corollary.

Corollary 1. For the weighted number of late jobs problem, there is an optimal schedule where the on-time jobs within each batch are ordered by the earliest due date rule.

Theorem 1 allows us to fix the order of the jobs within each batch for several scheduling criteria because there is an optimal schedule for the entire problem which preserves these batch orders. We define the *ordered batch scheduling problem* to be the problem of finding an optimal sum or max criterion schedule given that the order within each batch is fixed. Without loss of generality, we assume that the jobs in each batch are labeled according to this order.

A dynamic programming algorithm to solve the ordered batch scheduling problem is presented next. We define $C(n_1, n_2, \dots, n_B, t, a)$ to be the minimum cost of a partial schedule containing the first n_b jobs of each batch b , where the last job scheduled comes from batch a and is completed at time t . The initial values are $C(0, 0, \dots, 0, 0, 0) = 0$ and all other values are set to infinity. The optimal schedule cost is found by selecting the smallest value of the form $C(N_1, N_2, \dots, N_B, t, a)$ for some schedule completion time t , where

$$t \leq \sum_{b=1}^B \sum_{i=1}^{N_b} p_{ib} + \sum_{b=1}^B N_b \max_{0 \leq a \leq B} \{s_{ab}\},$$

and for some batch a to which the final job belongs. For $t > 0$ and $a > 0$, the costs can be computed

recursively using the equations

$$C(n_1, n_2, \dots, n_B, t, a) = \min_{0 \leq c \leq B} \{C(n'_1, n'_2, \dots, n'_B, t', c) + f_{n_a, a}(t)\} \quad (1)$$

where $+$ is the addition operator for the sum case, and the maximum operator for the max case. Furthermore, $n'_b = n_b$ for $b \neq a$ and $n'_a = n_a - 1$, and $t' = t - p_{n_a, a} - s_{ca}$.

In many cases, the large number of possible values of the state variable t may lead to computer storage problems. Alternatively, we can replace the state variable t by the variables t_{bc} , where $0 \leq b \leq B$, $1 \leq c \leq B$ and $b \neq c$, representing the number of setups from batch b to batch c in the partial schedule. Note that t is computed readily from the state variables using

$$t = \sum_{b=0}^B \sum_{c=1}^B t_{bc} s_{bc} + \sum_{b=1}^B \sum_{i=1}^{n_b} p_{ib}$$

A further reduction in storage can be achieved by choosing one state variable for each distinct s_{bc} value.

The following theorem gives the computational complexity of our dynamic programming algorithm. It shows that the algorithm is polynomially time bounded in the number of jobs but is exponentially time bounded in the number of batches. Since the ordering of jobs within batches requires $O(N \log N)$ time, the maximum lateness and total weighted completion time problems are polynomially solvable when the number of batches is fixed.

Theorem 2. *The ordered batch scheduling problem can be solved in $O(B^2 N^B \min\{N^S, T\})$ time for any sum or max criterion, where*

$$T = \sum_{b=1}^B \sum_{i=1}^{n_b} p_{ib} + \sum_{b=1}^B N_b \max_{0 \leq a \leq B} \{s_{ab}\}$$

and S is the number of different values for setup times. For sequence-independent setup times $S \leq B$, and in general $S \leq B^2 + B$.

Proof. Since T provides an upper bound on the maximum completion time of any schedule, there are $O(BN^B T)$ possible states for our dynamic programming equations, which include the state variable t . The value of each state is computed in $O(B)$ steps, thus giving one of the desired time bounds. When t is replaced, the number of possible states becomes $O(BN^{B+S})$, where the number of distinct setup times is bounded by $B^2 + B$ in general and by B in the case of sequence-independent setup times, thus establishing the alternative time bound.

The weighted number of late jobs problem is solved differently from the problems to which Theorem 1 applies. By Corollary 1, we only know an order for on-time jobs; those jobs that are to be on-time must also be determined by the algorithm. The late jobs may be appended in any order to the schedule of on-time jobs.

To derive our dynamic programming algorithm for the weighted number of late jobs problem, we first assume that the jobs within each batch are labeled according to earliest due date order. We define $C(n_1, n_2, \dots, n_B, t, a)$ to be the minimum weighted number of late jobs for a partial schedule containing the first n_b jobs of each batch b , where the last on-time job comes from batch a and is completed at time t . The initial values are

$$C(n_1, n_2, \dots, n_B, 0, 0) = \sum_{b=1}^B \sum_{i=1}^{n_b} w_{ib}$$

for $0 \leq n_b \leq N_b$, where $1 \leq b \leq B$, and all other values are set to infinity. The minimum weighted number of late jobs is found by selecting the smallest value of the form $C(N_1, N_2, \dots, N_B, t, a)$ for some completion time t of on-time jobs where

$$t \leq \max_{1 \leq b \leq B} \{d_{N_b, b}\}$$

and

$$t \leq \sum_{b=1}^B \sum_{i=1}^{N_b} p_{ib} + \sum_{b=1}^B N_b \max_{0 \leq a \leq B} \{s_{ab}\},$$

and for some batch a to which the final on-time job of the schedule belongs. For $t > 0$ and $a > 0$, the costs can be computed recursively using the equations

$$C(n_1, n_2, \dots, n_B, t, a) = \begin{cases} \min_{0 \leq c \leq B} \{C(n'_1, n'_2, \dots, n'_B, t', c)\} \\ C(n'_1, n'_2, \dots, n'_B, t, a) + w_{n_a, a} & \text{if } t \leq d_{n_a, a} \\ C(n'_1, n'_2, \dots, n'_B, t, a) + w_{n_a, a} & \text{if } t > d_{n_a, a} \end{cases} \quad (2)$$

where $n'_b = n_b$ if $b \neq a$ and $n'_a = n_a - 1$, and $t' = t - p_{n_a, a} - s_{ca}$. For $t \leq d_{n_a, a}$, job n_a of batch a is either on-time or late. The first term in the minimization corresponds to the case where job n_a of batch a is on-time and chooses a batch c for the previous on-time job; the second term corresponds to the case that job n_a of batch a is late. When $t > d_{n_a, a}$, job n_a of batch a cannot be on-time if it is completed at time t ; therefore, it is late. We observe that (2) is a direct

generalization of the recursion derived in Lawler and Moore (1969) for the case of a single batch.

As is the case in recursion 1, when processing or setup times are large, it is desirable to eliminate the state variable t from the recursion. To achieve this, we switch the state variable t with the function definition as follows. Define $C(n_1, n_2, \dots, n_B, w, a)$ to be the minimum completion time of on-time jobs for a partial schedule containing the first n_b jobs of each batch b , where the weighted number of late jobs is equal to w , and the last on-time job comes from batch a . The initial values are $C(n_1, n_2, \dots, n_B, w, 0) = 0$ when $w = \sum_{b=1}^B \sum_{i=1}^{n_b} w_{ib}$ for $0 \leq n_b \leq N_b$, where $1 \leq b \leq B$, and all other values are set to infinity. The minimum weighted number of late jobs is found by selecting the smallest w for which

$$\min_{0 \leq w \leq B} \{C(N_1, N_2, \dots, N_B, w, a)\}$$

is finite. The costs can be computed recursively using the equation

$$\begin{aligned} C(n_1, n_2, \dots, n_B, w, a) \\ = \min \left\{ \min_{0 \leq c \leq B} \{C(n'_1, n'_2, \dots, n'_B, w, c) + p'\} : \right. \\ \left. C(n'_1, n'_2, \dots, n'_B, w, c) + p' \leq d_{n_a, a} \right\} \\ C(n'_1, n'_2, \dots, n'_B, w', a) \} \quad (3) \end{aligned}$$

where $n'_b = n_b$ if $b \neq a$ and $n'_a = n_a - 1$, $p' = p_{n_a, a} + s_{ca}$ and $w' = w - w_{n_a, a}$. As in the recursion before the switch, the first term in the minimization chooses job n_a of batch a to be scheduled on-time if this is possible and chooses a batch c for the previous on-time job; the second term selects job n_a of batch a to be late.

The next theorem gives the computational complexity of our dynamic programming algorithms for the weighted number of late jobs problem. It shows that for a fixed number of batches, the time bound is polynomial for the number of late jobs problem and pseudopolynomial for the weighted number of late jobs problem.

Theorem 3. *The weighted number of late jobs problem can be solved in $O(B^2 N^B \min\{W, D, T\})$ time, where*

$$W = \sum_{b=1}^B \sum_{i=1}^{N_b} w_{ib},$$

$$D = \max_{1 \leq b \leq B} \{d_{N_b, b}\},$$

and

$$T = \sum_{b=1}^B \sum_{i=1}^{N_b} p_{ib} + \sum_{b=1}^B N_b \max_{0 \leq a \leq B} \{s_{ab}\}.$$

For the total number of late jobs problem, this time bound reduces to $O(B^2 N^{B+1})$.

Proof. For recursion 2 based on the values $C(n_1, n_2, \dots, n_B, t, a)$, since the completion time of on-time jobs is bounded above by $\min\{D, T\}$, there are $O(BN^B \min\{D, T\})$ states. For recursion 3 based on the values $C(n_1, n_2, \dots, n_B, w, a)$ there are $O(BN^B W)$ states. Since each recursion equation, (2) and (3), requires $O(B)$ steps to solve, the desired time bounds are obtained. For the number of late jobs problem, the unit weights yield $W = N$ to give a time bound of $O(B^2 N^{B+1})$.

In terms of the computational complexity of our dynamic programming algorithms, as given by Theorems 2 and 3, the number of late jobs problem is easier than the maximum lateness and total completion time problems. This is unusual in scheduling theory because it is generally the case that maximum lateness problems are easier than their counterparts in which the objective is to minimize the number of late jobs.

We conclude this section with a theorem that shows that the maximum lateness, and number of late jobs problems are NP-hard when the number of batches is arbitrary. This result indicates that it is unlikely that an algorithm with a time bound, which is not exponential in the number of batches, can be found.

Theorem 4. *The maximum lateness and the number of late jobs problems are NP-hard, even with sequence-independent setup times.*

Proof. The problem of determining whether a schedule exists with no late jobs is shown to be NP-complete in Bruno and Downey, even with sequence-independent setup times. This is clearly a special case of the maximum lateness and number of late jobs problems.

2. SCHEDULING IDENTICAL PARALLEL MACHINES

In this section, we consider the problem of scheduling N jobs that are divided into B batches on $M > 1$ identical parallel machines. Preemption is allowed, so the processing of any job may be interrupted and resumed at a later time on the same or another machine. When preemption occurs, the machine must be setup appropriately when the job is resumed.

We show that for two identical parallel machines, the maximum completion time problem is NP-hard.

This establishes that the maximum lateness and number of late jobs problems are NP-hard for $M = 2$.

Theorem 5. *The maximum completion time problem with two identical parallel machines is NP-hard, even with sequence-independent setup times.*

Proof. We show that an NP-complete problem is reducible to the maximum completion time problem. Our starting point is the following NP-complete problem.

Problem PARTITION. Given positive integers $a_1, \dots, a_r$ does there exist a subset $S \subset T = \{1, \dots, r\}$ such that $\sum_{i \in S} a_i = \sum_{i \in T-S} a_i$?

Given any instance of **PARTITION**, we define the following instance of the maximum completion time problem. Let $N = B = r$, $N_b = 1$ for $1 \leq b \leq r$, and $p_{1b} = a_b$ for $1 \leq b \leq r$ and $s_b = a_b$ for $1 \leq b \leq r$. Let $C_{\max}^*$ denote the minimum value of the maximum completion time.

We show that **PARTITION** has a solution if and only if $C_{\max}^* \leq A$, where $A = \sum_{i=1}^r a_i$. If **PARTITION** has a solution, then by scheduling the jobs of S on one machine and the jobs of $T - S$ on the other, the maximum completion time is A , thus showing that $C_{\max}^* \leq A$. We need to show that if $C_{\max}^* \leq A$, then **PARTITION** has a solution. Consider a schedule in which the job from some batch b is preempted. The total machine time required by such a schedule is at least $2A + a_b$, and therefore, has a maximum completion time that exceeds A . Thus, if $C_{\max}^* \leq A$, the optimal schedule is nonpreemptive. Let S denote the set of jobs scheduled on one of the machines. Clearly $\sum_{i \in S} a_i = A/2$ and $\sum_{i \in T-S} a_i = A/2$; otherwise $C_{\max}^* > A$. Therefore, the set S provides a solution of **PARTITION**.

For the nonpreemptive scheduling of two identical machines, the maximum completion time problem is NP-hard when all setup times are equal to zero (Bruno, Coffman and Sethi 1974). Thus, the corresponding maximum lateness and number of late job problems are also NP-hard.

We also deduce that the preemptive and nonpreemptive total weighted completion time problems are NP-hard for two identical parallel machines using the following observations. Firstly, the arguments of McNaughton (1959) may be used to show that there exists an optimal schedule without preemption. Secondly, Lenstra, Rinnooy Kan and Brucker (1977) show that the nonpreemptive weighted completion time problem with two identical machines is NP-hard when all setup times are equal to zero.

Finally, we observe that recursions 1 and 2 may be generalized to the nonpreemptive scheduling of identical parallel machines by the inclusion of time and batch state variables for each machine. This works because Theorem 1 can be applied to obtain an overall order that the jobs must respect on all the machines. This yields, for fixed B and M , pseudopolynomial algorithms for the maximum completion time, maximum lateness, total weighted completion time and weighted number of late jobs problems. However, such algorithms are mainly of theoretical interest because of the very high demands on computer storage.

3. CONCLUDING REMARKS

In this paper, we extend various scheduling models to include batch setup times and have attempted to classify these new problems as efficiently-solvable or NP-hard. We deduce from our dynamic programming algorithms that the one-machine maximum completion time, maximum lateness, total weighted completion time, and number of late jobs problems are efficiently-solvable when the number of batches is fixed even with sequence-dependent setup times. When the number of batches is a variable, the NP-hardness of the maximum lateness and number of late jobs is established, even when the setup times are sequence independent. The maximum completion time problem remains efficiently-solvable for sequence-independent setup times, but is NP-hard otherwise. The complexity of the total weighted completion time problem for arbitrary B and sequence-independent setup times is left as an open problem.

In the case of preemptively or nonpreemptively scheduling jobs from an arbitrary number of batches on two identical parallel machines, the maximum completion time, maximum lateness, number of late jobs and total weighted completion time problems are NP-hard; the complexity of the total completion time problem is left as an open problem.

The dynamic programming algorithms of Section 1 are largely of theoretical interest unless the number of batches is very small. An important research topic, therefore, is the design and analysis of heuristics for the various problems of scheduling with batch setup times. This would enable our models to be used in practical situations.

ACKNOWLEDGMENT

The research by the second author was partially supported by the Royal Society.

REFERENCES

- BRUNO, J., E. G. COFFMAN, JR. AND R. SETHI. 1974. Scheduling Independent Tasks to Reduce Mean Finishing Time. *Comm. ACM* **17**, 382–387.
- BRUNO, J., AND P. DOWNEY. 1978. Complexity of Task Sequencing With Deadlines, Set-Up Times and Changeover Costs. *SIAM J. Comput.* **7**, 393–404.
- CONWAY, R. W., W. L. MAXWELL AND L. W. MILLER. 1967. *Theory of Scheduling*. Addison-Wesley, Reading, Mass.
- LAWLER, E. L., J. K. LENSTRA, A. H. G. RINNOOY KAN AND D. B. SHMOYS. 1985. *The Traveling Salesman Problem*. John Wiley, New York.
- LAWLER, E. L., AND J. M. MOORE. 1969. A Functional Equation and its Application to Resource Allocation and Sequencing Problems. *Mgmt. Sci.* **16**, 77–84.
- LENSTRA, J. K., A. H. G. RINNOOY KAN AND P. BRUCKER. 1977. Complexity of Machine Scheduling Problems. *Ann. Discrete Math.* **1**, 343–362.
- MCNAUGHTON, R. 1959. Scheduling With Deadlines and Loss Functions. *Mgmt. Sci.* **6**, 1–12.
- MONMA, C. L., AND J. B. SIDNEY. 1979. Sequencing With Series-Parallel Precedence Constraints. *Math. Opns. Res.* **4**, 215–224.
- MONMA, C. L., AND J. B. SIDNEY. 1987. Optimal Sequencing via Modular Decomposition: A Characterization of Sequencing Functions. *Math. Opns. Res.* **12**, 22–31.
- SAHNEY, V. K. 1972. Single-Server, Two-Machine Sequencing With Switching Time. *Opns. Res.* **20**, 24–36.